

Full Stack Development (FSD) - Project Documentation

1. Introduction

- **Project Title:** Book a Doctor - Full Stack MERN Healthcare Booking Platform
 - **Team Members & Roles:**
 - **Backend Engineer:** Architecture design, JWT sessions, Express REST APIs, Mongoose validation, Multer files configuration.
 - **Frontend UI/UX Developer:** Bootstrap 5 responsive layout, context state management, dark/light theme, user dashboards.
-

2. Project Overview

- **Purpose:** The portal establishes a secure digital pathway between patients, medical practitioners, and administrators to facilitate doctor search, calendar scheduling, digital health locker storage, and prescription auditing.
 - **Key Features:**
 - *Patient:* Filtering doctor directories, booking slots with double-booking checks, uploading PDFs/images of past medical records.
 - *Doctor:* Slot coordination, approving/rejecting booking requests, writing digital prescriptions.
 - *Admin:* System stats diagnostics, vetting pending doctor credentials, managing platform users.
-

3. Architecture

The app follows a **3-tier Architecture** with MVC layout patterns separating user interaction, business controllers, and data storage.

- **Frontend:** Built with React 18, React Router DOM, Context API, Bootstrap 5, and custom glassmorphic styling.

- **Backend:** Developed with Node.js and Express.js, featuring Role-Based Access Control (RBAC) middlewares.
 - **Database:** MongoDB Atlas cloud cluster with Mongoose ODM modeling user, doctor, appointment, report, and notification schemas.
-

4. Setup Instructions

- **Prerequisites:** Node.js (v16+ recommended), npm, and a MongoDB instance (local or Atlas cluster).
- **Installation Steps:**
 1. Clone the repository and enter the root folder.
 2. Configure backend:

```
cd backend  
npm install
```

3. Create a `backend/.env` file:

```
PORT=5000  
MONGODB_URI=mongodb+srv://abhinayamodem_db_user:DoctorSecurePass123@cluster0.uzu  
JWT_SECRET=yoursecretaccesskey  
JWT_REFRESH_SECRET=yoursecretrefreshkey  
JWT_ACCESS_EXPIRY=15m  
JWT_REFRESH_EXPIRY=7d  
UPLOAD_PATH=uploads
```

4. Seed Mock Data:

```
npm run seed
```

5. Configure frontend:

```
cd ../frontend  
npm install
```

5. Folder Structure

```
book-a-doctor/
├── backend/
│   ├── config/           # DB & upload configuration
│   ├── controllers/      # Express route logic (auth, appointment, doctor, admin)
│   ├── middleware/       # Auth filters, file upload validations, error catchers
│   ├── models/           # Mongoose schemas (User, Doctor, Appointment, Report)
│   ├── routes/           # Express path routes
│   ├── utils/            # Token helpers and seeder scripts
│   └── uploads/          # Local document folder
├── frontend/
│   ├── src/
│   │   ├── components/   # Navbar, Protectors, Modals
│   │   ├── context/      # Auth & Theme context states
│   │   ├── pages/        # Login, Register, Dashboards, Listings
│   │   ├── App.js        # Router maps
│   │   └── index.js       # DOM render mount
│   └── package.json
└── README.md
```

6. Running the Application

To run the system locally, open two terminal windows (CMD):

- **Terminal 1 (Backend API):**

```
cd backend
npm run dev
```

- **Terminal 2 (Frontend UI):**

```
cd frontend
npm start
```

7. API Documentation

Authentication Routes

- `POST /api/auth/register` - Registers a new patient or doctor.
- `POST /api/auth/login` - Authenticates credentials and returns a JWT access token.
- `POST /api/auth/logout` - Clears cookies and tokens.

Appointment Routes

- `POST /api/appointments/book` - Places a scheduling request with double-booking check.
- `PATCH /api/appointments/:id/status` - Doctor approves or rejects booking requests.
- `POST /api/appointments/:id/prescribe` - Doctor completes appointment and registers prescription details.

Medical Locker Routes

- `POST /api/patient/upload` - Uploads past reports (only PDF/Images permitted).
-

8. Authentication

- **JWT Access Tokens:** Transmitted via HTTP headers (`Authorization: Bearer <token>`) for fast stateless route validation.
 - **JWT Refresh Tokens:** Stored in HttpOnly cookies, allowing users to refresh their access tokens silently without XSS exposure.
 - **RBAC:** Middleware checks incoming user records (`req.user.role`) and rejects requests if roles do not match permission schemas.
-

9. User Interface

- **Theme Engine:** Toggles between light and dark modes by updating CSS root variables dynamically.
 - **Dashboards:**
 - *Patient View:* Cards showcasing doctor directories and appointment logs.
 - *Doctor View:* Calendar schedule details and vetting lists.
 - *Admin View:* Diagnostic counts and approvals queue.
-

10. Testing

- **Unit & Integration Testing:** Express controllers are tested using **Jest** and **Supertest** assertions.
 - **Manual Testing:** Verified appointment workflows, verification queues, and locker upload limits.
-

11. Screenshots or Demo

- **Patient Dashboard:** `/patient/dashboard`
 - **Doctor Schedule Controller:** `/doctor/dashboard`
 - **Vetting Approval Center:** `/admin/dashboard`
 - **Active Prescriptions View:** `/patient/locker`
-

12. Known Issues

- **EADDRINUSE (Port 5000):** Occurs if ghost node processes remain active. Solved by taskkilling the conflicting process.
 - **DB Authentication Errors:** Occurs if special characters or spaces are included in Atlas configurations. Solved by setting database user passwords.
-

13. Future Enhancements

- **Video Call Consultations:** Integrating WebRTC.
- **Third-Party Payments:** Adding secure billing modules (Stripe/Razorpay).
- **Dynamic Reminders:** Incorporating SMS or email slot alerts.